

```
=====
                        SELECTION SORT ALGORITHM
                        in C Language
=====
```

PROGRAM DESCRIPTION:

Selection Sort is a simple sorting algorithm that divides the array into two parts: sorted and unsorted. It repeatedly finds the minimum element from the unsorted part and places it at the end of the sorted part.

```
=====
                        SOURCE CODE
=====
```

```
#include <stdio.h>

int main() {
    int arr[100], n, i, j, min, temp;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for (i = 0; i < n - 1; i++) {
        min = i;

        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min]) {
                min = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
=====
                        PROGRAM EXPLANATION
=====
```

1. ARRAY DECLARATION AND INPUT:
 - Declares array of maximum 100 elements
 - Takes number of elements from user
 - Reads all elements
2. OUTER LOOP (Sorted Portion):

```
for (i = 0; i < n - 1; i++)
```

 - Iterates from first element to second last
 - Expands sorted portion by one element each iteration
 - min = i initializes minimum index
3. INNER LOOP (Finding Minimum):

```
for (j = i + 1; j < n; j++)
- Searches unsorted portion for minimum
- Compares each element with current minimum
- Updates min when smaller element is found
```

4. SWAPPING:

- Exchanges minimum element with element at position i
- Moves minimum to sorted portion
- temp variable used for safe swapping

5. OUTPUT:

- Prints sorted array

ALGORITHM STEPS

Selection Sort Process:

Step 1: Initialize i = 0 (start of unsorted portion)

Step 2: Repeat until i = n-1:

- a) Set min = i
- b) Search from i+1 to n-1 for minimum element
- c) If smaller element found, update min
- d) Swap arr[i] with arr[min]
- e) Increment i (expand sorted portion)

Key Characteristic:

After i-th iteration, smallest i+1 elements are in sorted position.

EXAMPLE WALKTHROUGH

Array: [64, 34, 25, 12, 22]

Goal: Sort in ascending order

Initial State:

Sorted: [] Unsorted: [64, 34, 25, 12, 22]

Pass 1 (i=0):

Find min in [64, 34, 25, 12, 22]

min_index = 3 (value = 12)

Swap arr[0] and arr[3]

Array: [12, 34, 25, 64, 22]

Sorted: [12] Unsorted: [34, 25, 64, 22]

Pass 2 (i=1):

Find min in [34, 25, 64, 22]

min_index = 3 (value = 22)

Swap arr[1] and arr[3]

Array: [12, 22, 25, 64, 34]

Sorted: [12, 22] Unsorted: [25, 64, 34]

Pass 3 (i=2):

Find min in [25, 64, 34]

min_index = 2 (value = 25)

No swap needed (already at position)

Array: [12, 22, 25, 64, 34]

Sorted: [12, 22, 25] Unsorted: [64, 34]

Pass 4 (i=3):

Find min in [64, 34]

min_index = 4 (value = 34)

```
Swap arr[3] and arr[4]
Array: [12, 22, 25, 34, 64]
Sorted: [12, 22, 25, 34]    Unsorted: [64]
```

Final Sorted Array: [12, 22, 25, 34, 64]

COMPLEXITY ANALYSIS

Time Complexity:

Best Case: $O(n\hat{A}^2)$ - Comparisons same regardless of initial order
Average Case: $O(n\hat{A}^2)$ - Must check all remaining elements
Worst Case: $O(n\hat{A}^2)$ - Always $n(n-1)/2$ comparisons

Space Complexity:

$O(1)$ - Constant extra space (only temp variable)

Detailed Breakdown:

Total Comparisons: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$
Total Swaps: 0 to $n-1$ (at most one per iteration)
 $= O(n\hat{A}^2) + O(n) = O(n\hat{A}^2)$

Comparison Table:

Algorithm	Best	Average	Worst	Space
Selection Sort	$O(n\hat{A}^2)$	$O(n\hat{A}^2)$	$O(n\hat{A}^2)$	$O(1)$
Bubble Sort	$O(n)$	$O(n\hat{A}^2)$	$O(n\hat{A}^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n\hat{A}^2)$	$O(n\hat{A}^2)$	$O(1)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n\hat{A}^2)$	$O(\log n)$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

ADVANTAGES & DISADVANTAGES

Advantages:

- Simple to understand and implement
- No extra space required (in-place sorting)
- Efficient for small datasets (≤ 100 elements)
- Minimal memory writes (at most $n-1$ swaps)
- Predictable behavior (always $O(n\hat{A}^2)$)
- Good for small arrays with expensive comparisons

Disadvantages:

- $O(n\hat{A}^2)$ time complexity (inefficient for large datasets)
- Not stable (equal elements may change order)
- Poor cache performance (random memory access)
- Much slower than QuickSort or MergeSort
- Not suitable for real-time systems
- Performance doesn't improve with partially sorted data

KEY CHARACTERISTICS

Stability:

Not Stable - Elements with equal values may change relative order

Example: [3a, 1, 3b, 2]

After sorting: [1, 2, 3b, 3a] (3b and 3a switched positions)

In-Place:

Yes - Requires only $O(1)$ extra space

Comparison-Based:

Yes - Uses comparisons between elements

Adaptive:

No - Performance doesn't improve with partially sorted data

Natural:

No - Doesn't naturally handle linked lists

PRACTICAL APPLICATIONS

When to Use Selection Sort:

1. Educational purposes (easy to understand)
2. Very small datasets (< 50 elements)
3. Memory is extremely limited
4. Data access is expensive (minimizes writes)
5. Sorting where swaps are costly
6. As part of hybrid sorting algorithms

When NOT to Use:

1. Large datasets (> 1000 elements)
2. Time-critical applications
3. Linked lists (better: Insertion Sort)
4. Partially sorted data (better: Insertion Sort)
5. Real-time systems

OPTIMIZATION TECHNIQUES

1. Double Selection Sort:

- Find both min and max in one pass
- Place them at ends
- Reduces iterations

2. Hybrid Approach:

- Use Selection Sort for small arrays
- Switch to QuickSort for larger arrays

3. Early Termination:

- If array becomes sorted early
- Check if any swaps occurred in a pass

Modified Example - With Swap Counter:

```
int swaps = 0;
for (i = 0; i < n - 1; i++) {
    min = i;
    for (j = i + 1; j < n; j++) {
        if (arr[j] < arr[min]) {
            min = j;
        }
    }
    if (min != i) {
        // swap and increment counter
        swaps++;
    }
}
```

=====

COMPARISON WITH OTHER SIMPLE SORTS

=====

Selection Sort vs Bubble Sort:

- Both $O(n^2)$, but Selection uses fewer swaps
- Bubble Sort: $O(n^2)$ swaps worst case
- Selection Sort: $O(n)$ swaps worst case
- Selection Sort better for expensive swaps

Selection Sort vs Insertion Sort:

- Both $O(n^2)$ average and worst case
 - Insertion Sort: $O(n)$ best case
 - Selection Sort: $O(n^2)$ best case
 - Insertion Sort better for nearly sorted data
- =====